

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

О Т Ч Е Т

Тема задания: «РЕАЛИЗАЦИЯ БД С ИСПОЛЬЗОВАНИЕМ СУБД
MONGODB. ЗАПРОСЫ К БАЗЕ ДАННЫХ »

Обучающийся: Данилова Айаана Васильевна, К3241

Санкт-Петербург, 2026

I. ВВЕДЕНИЕ В СУБД MONGODB. УСТАНОВКА MONGODB. НАЧАЛО РАБОТЫ СУБД.

Цель: овладеть практическими навыками установки СУБД MongoDB.

Практическое задание:

1. Установите MongoDB для обеих типов систем (32/64 бита).
2. Проверьте работоспособность системы запуском клиента mongo.
3. Выполните методы:
 - a) db.help()
 - b) db.help
 - c) db.stats()
4. Создайте БД learn.
5. Получите список доступных БД.
6. Создайте коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.
7. Просмотрите список текущих коллекций.
8. Переименуйте коллекцию unicorns.
9. Просмотрите статистику коллекции.
10. Удалите коллекцию.
11. Удалите БД learn.

Запуск консольного клиента:

```
PS C:\Users\Далаана> mongosh
Current Mongosh Log ID: 6a12f146bca435e29aabc113
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.3.2
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-05-22T00:30:11.932+03:00: Access control is not enabled for the database. Read and write access to dat
a and configuration is unrestricted
-----
test>
```

Выполнение методов:

<pre>test> db.help() Database class: getMongo getName</pre>	<pre>test> db.help Database class: getMongo getName</pre>	<pre>test> db.stats() { db: 'test', collections: Long('0'), views: Long('0'), objects: Long('0'), }</pre>
--	--	--

Создание бд и коллекции в ней посредством вставки документа:

```
test> use learn
switched to db learn
learn> show dbs
admin    112.00 KiB
config   96.00 KiB
local    40.00 KiB

learn> db.unicorns.insertOne({name: 'Aurora', gender: 'female'})
{ acknowledged: true, insertedId: ObjectId('6a12f2a4bca435e29aabc114') }
learn> show dbs
admin    112.00 KiB
config   108.00 KiB
learn     8.00 KiB
local    40.00 KiB
```

Просмотр коллекции:

```
learn> db.getCollectionNames()
[ 'unicorns' ]
```

Переименование коллекции:

```
learn> db.unicorns.renameCollection('cuteUnicorns')
{ ok: 1 }
learn> db.getCollectionNames()
[ 'cuteUnicorns' ]
```

Статистика коллекции:

```
learn> db.cuteUnicorns.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,auto_compaction=0,block_compressor=snappy,block_manager=default,cache_resident=false,checksum=0,disaggregated=(page_log=,storage_tier=none),encryption=(keyid=,key_length=,key_type=)'
  }
}
```

Удаление коллекции и бд:

```
learn> db.cuteUnicorns.drop()
true
learn> db.getCollectionNames()
[]
learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
learn> show dbs
admin    112.00 KiB
config   108.00 KiB
local    40.00 KiB
```

II. ВСТАВКА ДОКУМЕНТОВ В КОЛЛЕКЦИЮ. ВЫБОРКА ДАННЫХ ИЗ БД.

Цель: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Практическое задание 2.1.1:

1) *Создайте базу данных learn.*

2) *Заполните коллекцию единорогов unicorns:*

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600,
gender: 'm', vampires: 63});

db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450,
gender: 'f', vampires: 43});

db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight:
984, gender: 'm', vampires: 182});

db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender:
'm', vampires: 99});

db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'],
weight:550, gender:'f', vampires:80});

db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733,
gender: 'f', vampires: 40});

db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690,
gender: 'm', vampires: 39});

db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421,
gender: 'm', vampires: 2});

db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601,
gender: 'f', vampires: 33});

db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650,
gender: 'm', vampires: 54});

db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540,
gender: 'f'});
```

3) *Используя второй способ, вставьте в коллекцию единорогов документ:*

```
{name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm',
vampires: 165}
```

4) *Проверьте содержимое коллекции с помощью метода find.*

Решение:

Заполнение данных единорогами:

```

users> use learn
switched to db learn
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 54});
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});

DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a12f7ffbca435e29aabc11f') }
}
learn> db.unicorns.find();
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a12f7ffbca435e29aabc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a12f7ffbca435e29aabc117'),
    name: 'Unicrom',

```

Создание документа и его вставка:

```

learn> favuni = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165};
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> d.unicorns.insert(favuni);
ReferenceError: d is not defined
learn> db.unicorns.insert(favuni);
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a12f851bca435e29aabc120') }
}
learn> db.unicorns.find(favuni);
[
  {
    _id: ObjectId('6a12f851bca435e29aabc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]

```

Практическое задание 2.2.1:

- 1) Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.
- 2) Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.

Решение:

- 1)

```
learn> db.unicorns.find({ gender: { $in: ['f', 'm'] } }).sort({ name: 1 }).limit(3)
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a12f7ffbca435e29aabc11a'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a12f851bca435e29aabc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
```

2)

```
learn> db.unicorns.find({ gender: 'f', loves: 'carrot' }).limit(1)
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

Решение:

```
learn> db.unicorns.find({ gender: 'm'}, {loves: 0})
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc115'),
    name: 'Horny',
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a12f851bca435e29aabc120'),
    name: 'Dunx',
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

Решение:

Предыдущая команда `db.unicorns.find()`, то есть `Dunx` корректно отобразился первым.

```
{
  _id: ObjectId('6a12f851bca435e29aabc120'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
]
learn> db.unicorns.find().sort({$natural:-1})
[
  {
    _id: ObjectId('6a12f851bca435e29aabc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  ]
```

Практическое задание 2.1.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

Решение:

```
learn> db.unicorns.find({}, {loves: {$slice:[1, 1]}, _id:0});
[
  {
    name: 'Horny',
    loves: [ 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Unicrom',
    loves: [ 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  ]
```

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

Решение:


```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

Решение:

```
learn> db.unicorns.find({ gender: 'm', loves: { $all: [ 'grape', 'lemon' ] }, weight: { $gte: 500, $lte: 700 } })
[ { _id: 0 } ]
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

Решение:

```
learn> db.unicorns.find({ vampires: {$exists: false}});
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc11f'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> |
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

Решение:

```
learn> db.unicorns.find({}, {name: 1, loves: {$slice: [1, 1]}}).sort({name: 1});
[
  {
    _id: ObjectId('6a12f7ffbc435e29aabc116'),
    name: 'Aurora',
    loves: [ 'grape' ]
  },
  {
    _id: ObjectId('6a12f7ffbc435e29aabc11a'),
    name: 'Ayna',
    loves: [ 'lemon' ]
  }
]
```

III. ЗАПРОСЫ К БАЗЕ ДАННЫХ MONGODB. ВЫБОРКА ДАННЫХ. ВЛОЖЕННЫЕ ОБЪЕКТЫ. ИСПОЛЬЗОВАНИЕ КУРСОРОВ. АГРЕГИРОВАННЫЕ ЗАПРОСЫ. ИЗМЕНЕНИЕ ДАННЫХ.

Практическое задание 3.1.1:

1) *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",
populatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: [""],
mayor: {
  name: "Jim Wehrle"
}}

{name: "New York",
populatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}

{name: "Portland",
populatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
  party: "D"}}
```

- 2) *Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.*
- 3) *Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.*

Решение:

1)

```
learn> db.towns.insertMany([
  {name: "Punxsutawney ",
    populatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: [""],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {name: "New York",
    populatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"}
  },
  {name: "Portland",
    populatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"}
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a131075b7907c626aabc115'),
    '1': ObjectId('6a131075b7907c626aabc116'),
    '2': ObjectId('6a131075b7907c626aabc117')
  }
}
```

2)

```
learn> db.towns.find({'mayor.party': 'I'}, {_id: 0, name: 1, mayor: 1})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
```

3)

```
learn> db.towns.find({'mayor.party': {'$exists': false}}, {_id: 0, name: 1, mayor: 1})
[ { name: 'Punxsutawney ', mayor: { name: 'Jim Wehrle' } } ]
learn> |
```

Практическое задание 3.1.2:

1. Сформировать функцию для вывода списка самцов единорогов.
2. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
3. Вывести результат, используя `forEach`.

Решение:

1)

```
learn> findMales = function() {var males = db.unicorns.find({gender: 'm'}); null; return males;};
[Function: findMales]
learn> findMales()
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a12f7ffbca435e29aabc117'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
]
```

2)

```
learn> var cursor = findMales().sort(this.name).limit(2);
```

3)

```
learn> cursor.forEach(function(obj){printjson(obj)});
{
  _id: ObjectId('6a12f7ffbca435e29aabc115'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
```

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({weight:{$gte: 500, $lte: 600}}).count()
4
learn> |
```

Практическое задание 3.2.2:

Вывести список предпочтений.

Решение:

```
learn> db.unicorns.distinct('loves')
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> |
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

Решение:

```
learn> db.unicorns.countDocuments()
12
learn> db.unicorns.aggregate({$group: {_id: '$gender', count: {$sum: 1}}})
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
```

Практическое задание 3.3.1:

IV. Выполнить команду:

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

V. Проверить содержимое коллекции *unicorns*.

Решение:

- 1) Метод **.save()** был официально удален из драйверов MongoDB и **mongosh** в последних версиях (начиная с MongoDB 4.x/5.x). У меня версия 8.3.2.

```
learn> db.unicorns.insert({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a131d36b7907c626aabc118') }
}
```

- 2) Документ вставлен в конец коллекции:

```
vampires: 165
},
{
  _id: ObjectId('6a131d36b7907c626aabc118'),
  name: 'Barney',
  loves: [ 'grape' ],
  weight: 340,
  gender: 'm'
}
```

Практическое задание 3.3.2:

1. Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
2. Проверить содержимое коллекции *unicorns*.

Решение:

- 1)

```
learn> db.unicorns.updateOne({name: 'Ayna'}, {$set:{weight: 800, vampires: 51}}, {upsert: true})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

- 2)

```
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc11a'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
```

Практическое задание 3.3.3:

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции unicorns.

Решение:

```
learn> db.unicorns.updateMany({name: 'Raleigh'}, {$push: {loves: 'redbull'}}, {upsert: true, multi: true})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc11c'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Практическое задание 3.3.4:

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.
2. Проверить содержимое коллекции unicorns.

Решение:

```
learn> db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}}, {upsert: true, multi: true})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find({gender: 'm'})
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a12f7ffbca435e29aabc117'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a12f7ffbca435e29aabc118'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  }
]
```

Практическое задание 3.3.5:

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции towns.

Решение:

```
learn> db.towns.updateOne({name: 'Portland'}, {$unset: {'mayor.party': 1}}, {})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a131075b7907c626aabc117'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Практическое задание 3.3.6:

1. Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.
2. Проверить содержимое коллекции unicorns.

Решение:

```
learn> db.unicorns.updateMany({name: 'Pilot'}, {$push: {loves: 'Chocolate'}}, {unset: true, multi: true})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'Chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

Практическое задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции unicorns.

Решение:

```
learn> db.unicorns.updateMany({name: 'Aurora'}, {$push: {loves: {$each: ['lemon', 'sugar']}}}, {unset: true, multi: true})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a12f7ffbca435e29aabc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'lemon', 'sugar' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Практическое задание 3.4.1:

1) Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",
```

```

    popujatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }}

    {name: "New York",
    popujatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"}}

    {name: "Portland",
    popujatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"}}

```

- 2) Удалите документы с беспартийными мэрами.
- 3) Проверьте содержание коллекции.
- 4) Очистите коллекцию.
- 5) Просмотрите список доступных коллекций.

Решение:

1)

```

learn> db.towns.find()
[
  {
    _id: ObjectId('6a131075b7907c626aabc115'),
    name: 'Punxsutawney ',
    popujatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ 'phil the groundhog' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a131075b7907c626aabc116'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a131075b7907c626aabc117'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]

```

2-3)

```

learn> db.towns.remove({'mayor.party': {'$exists': false}})
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a131075b7907c626aabc116'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]

```


4) Так как коллекция была очищена, а не удалена (drop), она отображается:

```
learn> db.towns.remove({})
{ acknowledged: true, deletedCount: 1 }
learn> db.getCollectionNames()
[ 'towns', 'users', 'unicorns' ]
learn> |
```

VI. ССЫЛКИ И РАБОТА С ИНДЕКСАМИ В БАЗЕ ДАННЫХ MONGODB

Практическое задание 4.1.1:

1. Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.
2. Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.
3. Проверьте содержание коллекции единорогов.

Решение:

1)

```
learn> db.kingdoms.insertMany([ {_id: 'Mur', name: 'Murino', description: 'I am fan of Melstroy'}, {_id: 'IL', name: 'IceLatteland', description: 'Ice Latte From Tiktok'}, {_id: 'LoL', name: 'Dolls world', description: 'Dlya adekvatnux poni'} ])
{ acknowledged: true,
  insertedIds: { '0': 'Mur', '1': 'IL', '2': 'LoL' } }
learn> |
```

2)

```
learn> db.unicorns.updateOne({name: 'Leia'}, {$set: {kingdom: {$ref: 'kingdoms', $id: 'Mur'}}}, {upsert: true})
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
learn> db.unicorns.updateOne({name: 'Nimue'}, {$set: {kingdom: {$ref: 'kingdoms', $id: 'LoL'}}}, {upsert: true})
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
learn> db.unicorns.updateOne({name: 'Ayna'}, {$set: {kingdom: {$ref: 'kingdoms', $id: 'IL'}}}, {upsert: true})
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
learn> |
```

3)

<pre>_id: ObjectId('6a1330d0b7907c626aabc123'), name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f', kingdom: DBRef('kingdoms', 'LoL')</pre>	<pre>_id: ObjectId('6a1330d0b7907c626aabc121'), name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33, kingdom: DBRef('kingdoms', 'Mur')</pre>
<pre>_id: ObjectId('6a1330d0b7907c626aabc11e'), name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40, kingdom: DBRef('kingdoms', 'IL')</pre>	

Практическое задание 4.2.1:

1. Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.
2. Содержание коллекции единорогов `unicorns`:

```
db.unicorns.insert({name: 'Horny', dob: new Date(1992,2,13,7,47),  
loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', dob: new Date(1991, 0, 24, 13, 0),  
loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', dob: new Date(1973, 1, 9, 22, 10),  
loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires:  
182});
```

```
db.unicorns.insert({name: 'Roooooodles', dob: new Date(1979, 7, 18, 18,  
44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', dob: new Date(1985, 6, 4, 2, 1),  
loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f',  
vampires:80});
```

```
db.unicorns.insert({name:'Ayna', dob: new Date(1998, 2, 7, 8, 30),  
loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires:  
40});
```

```
db.unicorns.insert({name:'Kenny', dob: new Date(1997, 6, 1, 10, 42),  
loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', dob: new Date(2005, 4, 3, 0, 57),  
loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', dob: new Date(2001, 9, 8, 14, 53),  
loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires:  
33});
```

```
db.unicorns.insert({name: 'Pilot', dob: new Date(1997, 2, 1, 5, 3),  
loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires:  
54});
```

```
db.unicorns.insert ({name: 'Nimue', dob: new Date(1999, 11, 20, 16,  
15), loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

```
db.unicorns.insert {name: 'Dunx', dob: new Date(1976, 6, 18, 18, 18),  
loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165
```

Решение:

Да, задать такой индекс можно, так как все текущие имена в коллекции уникальны, и MongoDB не обнаружит дубликатов, которые могли бы помешать созданию индекса.

```
learn> db.unicorns.ensureIndex({name: 1, unique: 1})  
[ 'name_1_unique_1' ]
```

Практическое задание 4.3.1:

1. Получите информацию о всех индексах коллекции `unicorns`.

2. Удалите все индексы, кроме индекса для идентификатора.
3. Попробуйте удалить индекс для идентификатора.

Решение:

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1, unique: 1 }, name: 'name_1_unique_1' }
]
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn>

learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.dropIndex('_id_')
MongoServerError[InvalidOptions]: cannot drop _id index
learn> |
```

Практическое задание 4.4.1:

1. Создайте объемную коллекцию *numbers*, задействовав курсор:
a. `for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}`
2. Выберите последние четыре документа.
3. Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра *executionTimeMillis*)
4. Создайте индекс для ключа *value*.
5. Получите информацию о всех индексах коллекции *numbers*.
6. Выполните запрос 2.
7. Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
8. Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

1)

```
learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1339f8b7907c626aad47d1') }
}
```

2)

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a1339f8b7907c626aad47d1'), value: 99999 },
  { _id: ObjectId('6a1339f8b7907c626aad47d0'), value: 99998 },
  { _id: ObjectId('6a1339f8b7907c626aad47cf'), value: 99997 },
  { _id: ObjectId('6a1339f8b7907c626aad47ce'), value: 99996 }
]
```

3)

```

executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 104,
  totalKeysExamined: 0,
  totalDocsExamined: 100000,
  executionStages: {
    isCached: false,
    stage: 'SORT',
  }
}

```

4-5)

```

learn> db.numbers.createIndex({ value: 1 })
value_1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]

```

6-7)

```

executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 2,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'LIMIT',
    nReturned: 4,
    executionTimeMillisEstimate: 0,
    works: 5,
    advanced: 4,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    limitAmount: 4,
    inputStage: {
      stage: 'FETCH',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 4,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 0,
      nss: 'learn.numbers',
      docsExamined: 4,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',

```

8) Благодаря индексу база данных перешла от полного сканирования всех 100 000 документов (**COLLSCAN**) к мгновенному поиску по готовому дереву индекса (**IXSCAN**). Это сократило количество просмотренных документов с 100 000 до всего лишь 4, что радикально уменьшило время выполнения с 104 мс до 2 мс.

Выводы

В ходе работы были освоены базовые методы установки и администрирования СУБД MongoDB, включая управление базами данных и создание коллекций. Использование агрегированных запросов и программных методов обработки курсоров продемонстрировало возможности автоматизации работы с записями в базе данных. Анализ планов выполнения запросов с помощью метода `explain` подтвердил, что использование индексов является ключевым фактором оптимизации производительности, радикально сокращая время поиска при работе с большими объемами информации.